To-Err-Is-to-LLM: Error Propagation and Selective Recovery in Multi-Agent Systems

GitHub: https://github.com/hrithikM86/CSE291A-To-Err-Is-to-LLM

All experiments reported in the final comparison were performed on a Mac with an Apple M5 chip and 16 GB RAM, using Python 3.11, MetaGPT, locally hosted Ollama models, and Docker-isolated test execution.

1 Architecture

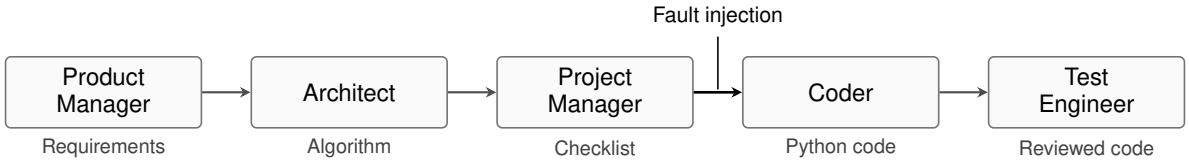

Figure 1: The five-agent sequential workflow. The fault is injected into the Project Manager's checklist before it reaches the Coder.

We evaluate 164 HumanEval Python problems using five sequential agents: Product Manager defines requirements, Architect designs the algorithm, Project Manager creates an implementation checklist, Coder writes code taking the checklist as a tab, and Test Engineer reviews and corrects it. All agents use qwen2.5:7b except the Coder, which uses qwen2.5-coder:7b. We extended the original dataset's test coverage with 504 additional test cases to check edge cases and evaluate both the Coder's code and the Tester's reviewed code alongside the official tests.

2 Fault Injection

The baseline runs all five agents without deliberate changes. Project Manager (PM) injection deliberately introduces a fault into the Project Manager's checklist before it is delivered to the Coder, while preserving the original problem specification. The Coder and Tester then rerun.

Example. HumanEval/0, has_close_elements. The task asks whether any two numbers differ by less than a threshold. The faulty PM checklist instead says to compare only the smallest and largest numbers. For [1.0, 1.1, 5.0] with threshold 0.2, the correct answer is True because 1.0 and 1.1 are close. Following the faulty checklist produces False. If the Coder follows this instruction and the Tester leaves it unchanged, the planning fault propagates into the final code.

3 Dependency Tracking

There's a tracker in place that records a run as a dependency graph. Every model call is one node (coder#1, coder#2), so the graph never has cycles even when agents call each other repeatedly. The task specification, each role's instructions and the model settings are nodes too. Calls are recorded by wrapping MetaGPT's model-provider methods (aask and related methods), so no agent call can bypass the tracker. Each dependency declared by the workflow's routing becomes an edge. The tracker compares the SHA-256 hash of the text the consumer received with the hash of the producer's actual output: equal is verified, difference is a mismatch. A mismatch flags a message altered in transit without the tracker being told where the fault came from. Finally, affected(node) returns every call that depends on a node, directly or indirectly, which is the set to recompute during selective recovery.

4 Recovery

R1 fixes the current instructions and reruns the affected agents. It keeps the Product Manager and Architect outputs, restores the original Project Manager checklist, and reruns the Coder and Tester. For example, HumanEval/0 (has_close_elements) asks whether any two numbers are close. The faulty checklist says "compare only the smallest and largest numbers." R1 replaces this with the original, correct checklist. We then vary what happens to the old faulty messages and code:

Drop deletes the old faulty messages and code, so the agents receive only corrected inputs. **Stale** keeps the earlier faulty work alongside the corrected instructions. **R3 Retract** replaces faulty history entries with "This previous message has been withdrawn." **R3 Both** deletes the faulty history and adds a notice telling the agents to disregard earlier faulty instructions.

Why is memory simulated? These agents do not automatically remember earlier runs. We deliberately add saved old messages to their inputs to imitate that memory. R3 handles this added history while R1 performs the recovery; R3 is not tested separately. The old history is reconstructed, but the agents generate new code and Docker tests it.

5 Results Across Baseline, Fault Injection, and Recovery

Tasks passing official tests count tasks that pass the original HumanEval test suite out of 164. Additional tests passed counts individual additional tests passed out of 504. Tasks passing all tests take into consideration the tasks that pass both the official suite and every additional test. Injection-broken tasks recovered determine how many of the 27 tasks that passed all tests before injection but failed afterward pass all tests again in the Tester's final reviewed code.

Table 1: Results across baseline, fault injection, and recovery.

Experimental condition	Agent	Tasks passing official tests	Additional tests passed	Tasks passing all tests	Injection-broken tasks recovered
Original baseline	Coder	128/164	458/504	124/164	—
Original baseline	Tester	133/164	466/504	130/164	—
PM injection	Coder	54/164	325/504	47/164	—
PM injection	Tester	118/164	437/504	114/164	—
R1 (Drop) – remove faulty history	Coder	128/164	458/504	124/164	—
R1 (Drop) – remove faulty history	Tester	133/164	466/504	130/164	27/27
R1 (Stale) – retain faulty history	Coder	80/164	379/504	75/164	—
R1 (Stale) – retain faulty history	Tester	119/164	443/504	113/164	7/27
R1 + R3 (Retract) – withdrawal markers	Coder	128/164	462/504	124/164	—
R1 + R3 (Retract) – withdrawal markers	Tester	131/164	464/504	128/164	27/27
R1 + R3 (Both) – remove history + notice	Coder	127/164	457/504	123/164	—
R1 + R3 (Both) – remove history + notice	Tester	129/164	459/504	126/164	25/27

5.1 Recovery Findings and Localisation Accuracy

PM injection reduced final-code passes on all tests from 130 to 114: it broke 27 previously passing tasks while improving 11 previously failing tasks. Drop recovered all 27 affected tasks and restored the baseline score of 130/164. Retract also recovered all 27 but achieved 128/164 overall, indicating failures elsewhere. Stale recovered only 7/27, while Both recovered 25/27.

Localisation accuracy was 164/164 (100%): saved message fingerprints and request records correctly identified the altered Project Manager →Coder handoff in every task. This measures localisation of the injected message change, not general detection of logically incorrect instructions. The restored PM text matched the original baseline on all 164 tasks in every recovery mode. A minor disclaimer over here would however be that the current test data set size is small which is why the localisation accuracy operates at extreme of 100%. The future scope entails adding more cases in order to test and validate robustness of our system.

5.2 Token Cost, Time and Efficiency

Token usage measures text processed, not monetary cost. Input tokens are sent to the model, while output tokens are generated by it. Efficiency is total tokens divided by recorded model-call time, expressed in tokens per second. Reported times sum model-call durations across the study; they do not represent the study's total elapsed time.

Each recovery makes 328 new model calls, using two calls per task instead of five. This reduces call count by 60% compared with rerunning the full pipeline, but does not imply equivalent savings in time or tokens. Higher token throughput also does not indicate better code quality.

Table 2: Token usage, model-call time and throughput.

Experimental condition	Agents	Input tokens	Output tokens	Total tokens	Model-call time (s)	Throughput (tokens/s)
Original baseline (reference for full recomputation)	All five agents	385,256	174,395	559,651	9,439.0	59.29
Original baseline	Coder + Tester	195,227	42,742	237,969	2,973.8	80.02
PM injection	Coder + Tester	166,043	40,958	207,001	2,772.5	74.66
R1 (Drop) – remove faulty history	Coder + Tester	195,227	42,742	237,969	4,877.0	48.79
R1 (Stale) – retain faulty history	Coder + Tester	328,048	41,936	369,984	3,030.2	122.10
R1 + R3 (Retract) – withdrawal markers	Coder + Tester	221,382	42,743	264,125	2,925.1	90.30
R1 + R3 (Both) – remove history + notice	Coder + Tester	211,219	46,612	257,831	3,163.3	81.51

A Future Directions and Proposed Timeline

The next phase will examine whether dependency-aware fault recovery remains effective beyond the current Project Manager to Coder intervention. The reported 100% localisation accuracy measures identification of recorded message changes at a known handoff. It does not establish detection accuracy for unknown semantic errors. The following extensions are proposed; their outcomes remain to be evaluated.

A.1 Detection Reliability and False Positives

Evaluate clean runs and harmless message changes to determine how often the system incorrectly triggers recovery. Report the false positive rate as the number of clean cases incorrectly flagged divided by the total clean cases evaluated. Distinguish detection of changed content from evidence that the change introduces an error.

A.2 Reducing Recovery Calls

Investigate when recovery can rerun only the Coder or Test Engineer instead of both. Compare the reduced-call approach with the current two-call procedure using final test performance, newly introduced failures, token usage, and runtime.

A.3 Recovery for Completed External Actions (Handling Irreversible Tool Calls)

Develop a small simulated workflow involving actions that cannot be undone simply by rerunning an agent, such as a completed payment. The system should explain the error and offer appropriate options, such as requesting a reversal or escalating to support. Corrective critical actions should require user authorisation, with their outcomes recorded and verified.

A.4 History Cleaning with Persistent Agent Memory

Extend R3 to agents that retain conversation history between calls. Implement removal and withdrawal markers for invalidated messages, then inspect subsequent model requests to verify that faulty content is absent or explicitly withdrawn. Compare recovery performance with the current simulated-history experiments.

A.5 Recovery Without a Verified Baseline Message

Generate a corrected Project Manager checklist from the original task specification when no verified baseline message is available. Evaluate whether the regenerated checklist restores downstream correctness without introducing new errors. Compare this approach with the current method of restoring a saved original checklist.

A.6 Fault Injection at Additional Agent Stages

Introduce task-specific faults into other stages, starting with the Architect or Test Engineer. Measure how fault location changes downstream failures and the stages requiring replay. Once single-fault experiments are established, investigate selected cases containing faults at more than one stage.

A.7 Evaluation Across Models, Different Tasks and Varied Architectural Structures

Evaluate more capable LLMs accessed through Amazon Bedrock, and extend our experiments beyond code generation to other multi-agent tasks. Move beyond the linear pipeline studied here to cyclic, hierarchical and other workflow structures. In these structures, agents revisit earlier stages and not every downstream computation depends on a given fault, which makes them a stronger test of selective recovery.

A.8 Proposed Timeline

Period	Planned work
Nov 3–9	Evaluate false positives and test fingerprints.
Nov 10–16	Pilot recovery without saved baseline messages.
Nov 17–23	Test recovery with fewer model calls plus addition of irreversible tool calls handling.
Nov 24–30	Prototype R3 with persistent agent memory.
Dec 1–7	Evaluation across models, different tasks and varied architectural structures.
Dec 8–10	Consolidate findings, document limitations, and finalise the report.

References

[1] Jen-tse Huang, Jiaxu Zhou, Tailin Jin, Xuhui Zhou, Zixi Chen, Wenxuan Wang, Youliang Yuan, Michael R. Lyu, and Maarten Sap. On the resilience of LLM-based multi-agent collaboration with faulty agents. In Proceedings of the 42nd International Conference on Machine Learning, PMLR 267, 2025.

[2] Caili Yu, Yiqi Wang, Jiaqi Zhang, Yiqun Duan, Mingkai Zheng, Zhangkai Wu, Kaize Shi, and Taotao Cai. From faulty memories to corrected actions: Dependency-guided rollback repair for memory-augmented agents. arXiv:2608.10502, 2026. https://arxiv.org/abs/2608.10502.